Domain Name Server

Task 1: Install BIND DNS server and verify installation

Step 1: Open the terminal as root

Switch to root:

su -

Enter your root password.

Step 2: Install the required packages

dnf install bind bind-utils -y

- bind — installs the DNS server.
- bind-utils — provides commands such as dig and nslookup.
- -y — automatically confirms installation.

If the installation fails because of a repository error, check your RHEL repository configuration rather than reinstalling Linux.

Step 3: Verify the version

named -v

Expected output will look similar to:

BIND 9.x.x (Extended Support Version)

The exact version may differ.

Step 4: Start and enable the DNS service

systemctl enable --now named

Check its status:

systemctl status named

You should see:

Active: active (running)

Press q to exit the status screen.

Task 1 is complete when BIND is installed, named -v displays its version, and the service is running.

Task 2: Configure a DNS zone for myzomatoclone.local

First, understand what we're creating

DNS translates domain names into IP addresses. In this assignment, we'll configure a local domain:

- Domain: myzomatoclone.local
- IP address: 127.0.0.1 (the local loopback address)
- Alias: foodsearch.myzomatoclone.local

Our goal is to make BIND answer queries for this custom domain.

Your RHEL Linux VM

DNS client sends a query

Query: myzomatoclone.local

BIND DNS server (named)

Reads the configured zone file

DNS answer: 127.0.0.1

Step 1: Understand the important files

On RHEL, we'll use these files:

File	Purpose
/etc/named.conf	Main DNS server configuration
/var/named/myzomatoclone.local.zone	Stores the DNS records for our domain

Your assignment mentions /etc/bind/ because that is common on Ubuntu/Debian. For RHEL, use /etc/named.conf and /var/named/.

Step 2: Add the zone configuration

Open the main configuration file:

nano /etc/named.conf

Go to the end of the file. Add the following zone definition outside the existing options { } block:

```
zone "myzomatoclone.local" IN {
    type master;
    file "myzomatoclone.local.zone";
    allow-query { localhost; };
};
```

Save with Ctrl+O, press Enter, and exit with Ctrl+X.

Explanation:

- zone — identifies the domain managed by BIND.
- type master — this server holds the original zone data.

- file — specifies the file containing DNS records.
- allow-query { localhost; }; — allows DNS queries from this same machine only.

Step 3: Create the zone file

On RHEL, create the zone file in /var/named/:

nano /var/named/myzomatoclone.local.zone

Enter this complete configuration:

```
$TTL 86400
@   IN SOA ns1.myzomatoclone.local. admin.myzomatoclone.local. (
        2026102001
        3600
        1800
        604800
        86400
)
@           IN NS      ns1.myzomatoclone.local.
ns1         IN A       127.0.0.1
@           IN A       127.0.0.1
foodsearch  IN CNAME   myzomatoclone.local.
```

Save and exit.

Understand each DNS record

Record	Meaning
$TTL 86400	Default record lifetime: 86,400 seconds (24 hours).
SOA	Start of Authority; contains administrative information about the zone.
NS	Specifies the authoritative DNS server for the zone.
A	Maps a hostname to an IPv4 address.
CNAME	Creates an alias for another hostname.

The @ symbol represents the zone name, myzomatoclone.local.

Therefore, this record:

@ IN A 127.0.0.1

maps myzomatoclone.local to 127.0.0.1.

This record:

foodsearch IN CNAME myzomatoclone.local.

creates the alias foodsearch.myzomatoclone.local.

Important: The final dot in myzomatoclone.local. means the target is a complete domain name. Without that dot, BIND can append the current zone name to the target and produce an unintended hostname.

Step 4: Set file permissions and validate the configuration

Run these commands as root:

chown root:named /var/named/myzomatoclone.local.zone

chmod 640 /var/named/myzomatoclone.local.zone

Now validate the main configuration:

named-checkconf

If there is no output, the configuration syntax is correct.

Validate the zone file:

named-checkzone myzomatoclone.local /var/named/myzomatoclone.local.zone

Expected result:

zone myzomatoclone.local/IN: loaded serial 2026102001

OK

If you see OK, the zone file has passed validation.

Step 5: Restart the DNS service

systemctl restart named

systemctl status named

Check that the status shows active (running).

If the service fails to start, check the error details:

journalctl -u named -n 30 --no-pager

Official RHEL guidance also recommends validating both the configuration and zone file before reloading BIND.

Red Hat Documentation

+1

Task 3: Set the DNS resolver and test domain resolution

A DNS server and a DNS resolver are related, but they have different roles:

- DNS server: BIND (named) stores and answers for your custom zone.
- DNS resolver configuration: tells your Linux machine which DNS server to query.

Step 1: Test BIND directly first

Run:

dig @127.0.0.1 myzomatoclone.local

Look for an answer similar to:

myzomatoclone.local. 86400 IN A 127.0.0.1

The important part is A 127.0.0.1.

You can also run the shorter command:

dig +short @127.0.0.1 myzomatoclone.local

Expected output:

127.0.0.1

The @127.0.0.1 part explicitly tells dig to query the DNS server running on your own VM.

Step 2: Test using nslookup

nslookup myzomatoclone.local 127.0.0.1

Expected output should include:

Name: myzomatoclone.local

Address: 127.0.0.1

If these tests work, your DNS zone is responding correctly.

Step 3: Configure the system resolver

First, check the active NetworkManager connection:

nmcli connection show --active

Example output:

NAME UUID TYPE DEVICE

Wired connection 1 example-uuid ethernet ens160

Your connection name may be different.

Set the local DNS server, replacing Wired connection 1 with your actual connection name:

nmcli connection modify "Wired connection 1" ipv4.dns "127.0.0.1"

nmcli connection modify "Wired connection 1" ipv4.ignore-auto-dns yes

nmcli connection up "Wired connection 1"

Caution: Reactivating the connection may briefly interrupt networking. If your VM relies on this connection for internet access, the setting above may stop external domain resolution because BIND has not been configured as a forwarding resolver.

For a basic assignment, you can skip this persistent change and use dig @127.0.0.1 ... to verify your zone. That avoids changing your normal internet DNS settings.

Check the resolver configuration:

cat /etc/resolv.conf

You should see nameserver 127.0.0.1 if the resolver is configured to use localhost.

Task 4: Add and verify the CNAME alias

The alias record is already included in the zone file:

foodsearch IN CNAME myzomatoclone.local.

If you didn't add it earlier, edit the zone file:

nano /var/named/myzomatoclone.local.zone

Add the CNAME on a separate line and increment the SOA serial number, for example, from 2026102001 to 2026102002.

Then validate and reload:

named-checkzone myzomatoclone.local /var/named/myzomatoclone.local.zone

systemctl reload named

Test the alias:

dig @127.0.0.1 foodsearch.myzomatoclone.local

Or use:

dig +short @127.0.0.1 foodsearch.myzomatoclone.local

Expected output:

myzomatoclone.local.

127.0.0.1

The exact output depends on the dig options. The key is that the alias ultimately resolves to 127.0.0.1.

Task 5: Find and fix the incorrect zone-file syntax

Your assignment provides this incorrect snippet:

myzomatoclone.local. IN A 127.0.0.1 foodsearch IN CNAME myzomatoclone.local.

What is wrong?

Two DNS records have been written on the same line. BIND expects each resource record to be a separate entry.

Correct version

myzomatoclone.local. IN A 127.0.0.1

foodsearch IN CNAME myzomatoclone.local.

The first record maps the main domain to an IP address. The second record creates the alias.

Another important rule: A CNAME record should not be used alongside other record types for the same alias name. For example, don't create both an A record and a CNAME record for foodsearch in the same zone.

After fixing the file, run:

named-checkconf

named-checkzone myzomatoclone.local /var/named/myzomatoclone.local.zone

systemctl reload named

Finally, verify both records:

dig +short @127.0.0.1 myzomatoclone.local

dig +short @127.0.0.1 foodsearch.myzomatoclone.local

Both lookups should ultimately return 127.0.0.1.

Common errors and quick fixes

Error	What to check
command not found: dig	Run dnf install bind-utils -y.
connection refused	Check systemctl status named and whether BIND listens on localhost.
SERVFAIL	Run named-checkconf, named-checkzone, and inspect journalctl -u named.
NXDOMAIN	Check the zone name, hostname spelling, and whether the zone was reloaded.
permission denied	Check ownership and permissions of the zone file.
Alias does not resolve correctly	Ensure the CNAME is on its own line and its target ends with a dot.

One final note: .local is commonly used for multicast DNS on some systems, so ordinary system lookups can behave differently from direct BIND queries. For this lab, use dig @127.0.0.1 ... to test BIND directly.